Python Functions: Reusable Code

Write code once, use it again and again.

Python | Beginner to Intermediate | Ages 10–16

InfiBee Coders

The Big Idea

- A function is a named block of code that you can run whenever you need it.
- Functions make programs shorter, tidier and easier to fix.
- You have already used functions like `print()`, `len()` and `input()`.
- You make your own with the word `def`.

Anatomy of a Function

- `def name(parameters):` starts the definition. Indent the body by 4 spaces.
- A parameter is a name inside the brackets. An argument is the real value you pass in.
- `return` sends a value back. `print` only shows it on the screen.
- A function does nothing until you call it: `greet("Mia")`.
- If there is no `return`, the function gives back `None`.

Parameters in Detail

- Default value: `def area(w, h=1):` lets you skip `h`.
- Keyword arguments: `area(w=4, h=5)` use names so order does not matter.
- You can return many values: `return a, b` gives a tuple.
- A function can call other functions.

Scope

- A variable made inside a function is local. It does not exist outside.
- A variable made outside is global. Functions can read it.
- Prefer passing values in and returning results out, instead of using global variables.

💻 Code Examples

A function that prints python

```python
def say_hello():
    print("Hello!")
    print("Welcome to Python")

say_hello()
say_hello()
```

▶ **Output:**

```
Hello!
Welcome to Python
Hello!
Welcome to Python
```

Defining does nothing. Each call runs the body.

Parameters and return python

```python
def greet(name):
    return "Hello, " + name + "!"

message = greet("Mia")
print(message)
print(greet("Zoya"))
```

▶ **Output:**

```
Hello, Mia!
Hello, Zoya!
```

greet gives back a string that we can store or print.

Default parameter python

```python
def area(width, height=1):
    return width * height

print(area(4, 5))
print(area(7))
print(area(height=3, width=2))
```

▶ **Output:**

```
20
7
6
```

When height is missing, Python uses the default 1.

Return many values python

```python
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([5, 2, 9, 4])
print(low, high)
```

▶ **Output:**

```
2 9
```

The two returned values are unpacked into two variables.

Scope: local vs global python

```python
points = 10

def add_bonus():
    bonus = 5
    return points + bonus

print(add_bonus())
print(points)
```

▶ **Output:**

```
15
10
```

bonus lives only inside the function.

A function with a decision python

```python
def grade(mark):
    if mark >= 90:
        return "A"
    elif mark >= 75:
        return "B"
    return "Keep going"

print(grade(95), grade(80), grade(40))
```

▶ **Output:**

```
A B Keep going
```

return ends the function straight away.

Forgetting an argument python

```python
def greet(name):
    return "Hi " + name

print(greet())
```

▶ Error message:

```
TypeError: greet() missing 1 required positional argument: 'name'
```

Fix: give the function the value it needs.

📚 Key Terms

Function
A named block of reusable code.

def
The word that creates a function.

Parameter
A name in the brackets of a function definition.

Argument
The real value you pass into a function.

Call
Running a function by using its name.

return
Sends a value back from a function.

Default value
A parameter value used when none is given.

Scope
Where a variable can be used.

Local variable
A variable that exists only inside a function.

None
The value of nothing, returned when there is no return.

🧮 Quick Reference

```
def name(a, b=1):
    return a + b

name(5)          # call
```

```
name(5, 2)       # two args
name(b=2, a=5)   # by keyword
x, y = f()       # two returns
```

✏️ Worked Examples

Q: Write a function that doubles a number.

A: def double(n): return n * 2. Then print(double(6)) shows 12.

Q: Write a function that checks if a number is even.

A: def is_even(n): return n % 2 == 0. is_even(4) gives True and is_even(7) gives False.

Q: Why does a function print None sometimes?

A: It used print inside but had no return. Python returns None when there is no return statement.

🚫 Common Mistakes

- Calling a function without brackets: `greet` instead of `greet("Mia")`.
- Using `print` inside when you needed `return`, so the result is `None`.
- Forgetting the colon after the `def` line.
- Writing code after `return`, which never runs.
- Using a variable from inside the function outside it.

👨‍👩‍👧 Parent Corner

- Compare a function to a recipe card: write it once and use it whenever you cook.
- Ask your child to spot repeated code in their program and turn it into a function.
- Ask: What goes in? What comes out?
- Encourage short functions that do one job and have clear names.
- Have them test with easy values first, like 0 and 1.

🎯 Quick Check

1. What word creates a function?

✅ def.

2. What is the difference between a parameter and an argument?

✅ A parameter is the name in the definition. An argument is the value you pass in when calling.

3. What does return do?

✅ Sends a value back to the code that called the function.

4. What does a function without return give back?

✅ None.

5. What does def greet(name): do on its own?

✅ It defines the function. Nothing runs until you call it.